第3章 指令系统

3.0 概述

3.1 寻址方式

3.2 MCS-51指令的分类

3.3 数据传送指令

3.4 逻辑运算指令

3.5 算术运算指令

3.6 控制转移指令

3.7 汇编语言、伪指令

第3章 指令系统

3．0 概述

指令系统：计算机所能识别和执行的所有指令的集合。

①	②	③	④	⑤
操作码	第一个操作数地址	第二操作数或地址	结果数地址	下一条指令地址

MCS-51总共有111条指令：

字 长		时 间	
单字节指令	49条	1个机器周期指令	64条
双字节指令	45条	2个机器周期指令	45条
三字节指令	17条	4个机器周期指令	2条

举例:

1、单字节指令 MOV R0, A ; (R0) ← A

D7	D6	D5	D4	D3	D2	D1	D0
1	1	1	1	1	0	0	0

F8H

2、 双字节指令 MOV 20H, R0 ; (20H) ← (R0)

D7	D6	D5	D4	D3	D2	D1	D0
1	0	0	0	1	0	0	0
0	0	1	0	0	0	0	0

操作码 88H

操作数 20H

3、三字节指令 MOV 60H，#2FH ； （60H）← 2FH

D7	D6	D5	D4	D3	D2	D1	D0	
0	1	1	1	0	1	0	1	操作码 75H
0	1	1	0	0	0	0	0	操作数地址60H
0	0	1	0	1	1	1	1	操作数 2FH

3.1 寻址方式

3.1.1 汇编语言指令格式

3.1.2 寻址方式

在指令中给出取操作数或存放结果的方式。

在MCS-51系统中有7种寻址方式:

1、立即寻址
2、直接寻址
3、寄存器寻址
4、寄存器间接寻址
5、变址寻址
6、相对寻址
7、位寻址

3.1.1 汇编语言指令格式

汇编语言：用助记符书写的机器语言。

汇编语言格式为：

[标号：] 操作码助记符 [目的操作数，] [源操作数] [；注释]

如：STAR： MOV A， #01H ； (A)=01H

标号 操作码 操作数 注释

标号：给该条语句起个名字，以便在其它语句中寻找该条语句。它代表该条语句所在的地址。它由1~6个字符数字串组成，第一个必须是字母，用冒号隔开。不一定每句都有标号，它不产生目标代码。

操作码：指令的助记符。说明指令执行操作类型。

操作数：说明被操作的数的源及目的。它可以是数的本身或其所在地址。有两个以上操作数时可称为第一操作数、第二…。

书写要求完整，注意标点符号及数的后缀。十六进制数以A~F打头的数前面要加0，以区别于标号。例如十六进制数A5H应写0A5H。

注释：用简单的文字或符号说明该条指令的操作，注释段不产生目标代码。用分号开始表示注释。

汇编语言程序说明格式：

# data8、 # data16:	；8位或者16位的立即数
Direct:	；直接地址单元
SFR：	；特殊功能寄存器
Rn(n=0-7):	；通用寄存器
@ Ri(i=0、1）：	；间接寻址寄存器
Pi (i=0-3):	；四个并行I/O口
Bit:	；位操作数或者位地址
Addr11 、addr16	；11位或16位目的地址
（X）	;表示X地址单元或寄存器中的内容
（（X））	;表示以X地址单元或寄存器中的内容作为地址来间接寻址单元的内容

3.1.2 寻址方式

一条指令的寻址方式，指的是*源操作数*的寻址方式

1. 寄存器寻址

是对由指令选定的工作寄存器（R0-R7）进行读/写；

如：

MOV A，Rn；

在指令中指定了从寄存器Rn(0-7)中取得源操作数，所以称为：寄存器寻址方式

2. 直接寻址

是指由指令直接给出操作数的单元地址。

MOV A, direct;

MOV direct1, direct2;

可访问三种地址空间：

- SFR空间，**唯一能访问SFR的寻址方式**；
- 是访问片内RAM单元的一种寻址方式；
- 位地址空间；

如：MOV A, 50H

操作数直接存放在内部RAM低地址的128字节，或者是特殊功能寄存器。

注意：特殊功能寄存器算是直接寻址。

3. 寄存器间接寻址

是把由指令所指定的寄存器内容作为地址，由该地址所指定的单元内容作为操作数，操作数可以从该单元读出或写入。

MCS-51规定：R0或R1作间接寻址寄存器，用以寻址内部或外部数据RAM 256个字节单元；

还可以采用数据指针DPTR作间接寻址寄存器，寻址外部数据RAM的64K字节单元。

如：MOV A,@ R1

操作数存放在数据存储器中，存储单元地址存放在某个寄存器中，可以用间接寻址的寄存器有R0、R1、DPTR三个，注意一定不能忘了@。

寄存器间接寻址

- 如：MOV A，@R0

@表示间接寻址指示符

(R0)=65H，(65H)=47H,

执行该指令后，（A）=?

注意区分：　　MOV　A，@R0；片内RAM

　　MOVX　A，@R0；片外RAM

两条指令源及目的相同，而操作域不同，以助记符区分。

4. 立即寻址

是由指令中直接给出操作数。

如：MOV A，＃30H

直接给出操作数的寻址方式，前面用＃表示立即数。立即数可以是十进制，也可以是十六进制，还可以二进制。

注意：立即数只能作为原操作数。

如：MOV A,＃10000111B

MOV R1,＃50

MOV R1,#0FDH

5. 位寻址

如：MOV C，P1.1

主要包含可位寻址区的内部RAM和能被8整除的SFR。

下表为位寻址区的位地址。

位 地 址 表

字节地址	D7	D6	D5	D4	D3	D2	D1	D0
2FH	7FH	7EH	7DH	7CH	7BH	7AH	79H	78H
2EH	77H	76H	75H	74H	73H	72H	71H	70H
2DH	6FH	6EH	6DH	6CH	6BH	6AH	69H	68H
2CH	67H	66H	65H	64H	63H	62H	61H	60H
2BH	5FH	5EH	5DH	5CH	5BH	5AH	59H	58H
2AH	57H	56H	55H	54H	53H	52H	51H	50H
29H	4FH	4EH	4DH	4CH	4BH	4AH	49H	48H
28H	47H	46H	45H	44H	43H	42H	41H	40H
27H	3FH	3EH	3DH	3CH	3BH	3AH	39H	38H
26H	37H	36H	35H	34H	33H	32H	31H	30H
25H	2FH	2EH	2DH	2CH	2BH	2AH	29H	28H
24H	27H	26H	25H	24H	23H	22H	21H	20H
23H	1FH	1EH	1DH	1CH	1BH	1AH	19H	18H
22H	17H	16H	15H	14H	13H	12H	11H	10H
21H	0FH	0EH	0DH	0CH	0BH	0AH	09H	08H
20H	07H	06H	05H	04H	03H	02H	01H	00H

28H	47H	46H	45H	44H	43H	42H	41H	40H
27H	3FH	3EH	3DH	3CH	3BH	3AH	39H	38H
26H	37H	36H	35H	34H	33H	32H	31H	30H
25H	2FH	2EH	2DH	2CH	2BH	2AH	29H	28H
24H	27H	26H	25H	24H	23H	22H	21H	20H
23H	1FH	1EH	1DH	1CH	1BH	1AH	19H	18H
22H	17H	16H	15H	14H	13H	12H	11H	10H
21H	0FH	0EH	0DH	0CH	0BH	0AH	09H	08H
20H	07H	06H	05H	04H	03H	02H	01H	00H

如：下列指令的操作功能是一样的

```
MOV  24H.1, C ;
MOV  21H, C;
```

注意：不能用这样的指令：

```
MOV  24H.1, 1
```

位传送时，必须有一个操作数为 C 。

6. 变址间接寻址

基址寄存器内容　加　变址寄存器内容之和，作为有效地址进行间接寻址的方式，可实现动态寻址。

如：

```
MOVC A,@A＋DPTR;
MOVC A, @A+PC
JMP @A+DPTR
```

其中：DPTR作为基地址，A中内容作为偏移地址，相当于把两个寄存器中的地址相加，把相加后的地址单元中的内容送到A寄存器中。

基地址寄存器：程序计数器PC或数据指针DPTR。

变址寻址主要用于查表用。

7. 相对寻址

是以PC的当前值为基准，加上指令中给出的偏移量（rel）形成新的有效转移地址。

如：SJMP REL

前面我们说了，每条指令在程序存储器中都有其地址;相对寻址，是指在当前指令地址的前提下进行寻址，如上例中，假设指令存放于2030H单元中，REL＝30H，那么程序跳到2062H单元去了。 SJMP 是2字节指令。

注意，相对偏移量（rel）是一个带符号数的8位二进制数，其最高位为符号位，所以其转移范围为：

以PC的当前值为起始值，相对偏移在-128~+127个字节之间。偏移量必须以补码形式出现在指令中。

寻址方式小结

1 寄存器寻址: R0--R7，Acc，B，Cy，DPTR

2 直接寻址: 内部RAM低地址的128字节和SFR

3 寄存器间接寻址: 内部RAM（@R1,@R0,SP),外部数据RAM（@R1,@R0,@DPTR)

4 立即寻址：程序存储器立即数

5 变址间接寻址：程序存储器（@A+DPTR,@A+PC）

6 相对寻址：以PC当前值为基地址+指令中给出的偏移量=有效转移地址；PC当前值的-128~+127。

7 位寻址：对内部RAM或SFR的可位寻址单元进行寻址

3.2 MCS-51指令的分类

共有111条指令，按其功能分类：

1. **数据传送指令 共28条；**
2. **算术逻辑运算指令 共49条；**
   其中算术24条、逻辑25条；
3. **控制转移指令 17条；**
4. **位操作指令 17条。**

3.3 数据传送指令

- 1. MOV指令
- 2. 栈操作指令
- 3. 交换指令
- 4. 程序存储器传送指令　MOVC
- 5. 片外数据存储器RAM传送操作　　MOVX

3.3 数据传送指令

数据传送指令的功能：进行数据传送。

包括如下几类指令：

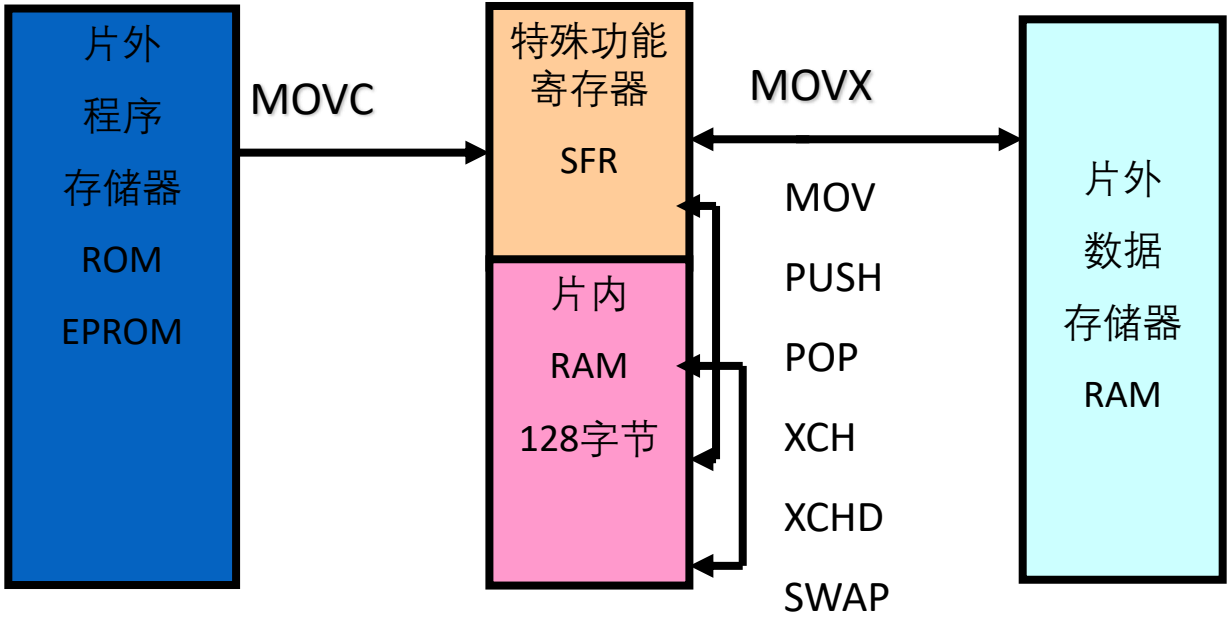
★片内RAM传送指令：　　MOV

栈操作指令：　　PUSH　POP

交换指令：　　SWAP　XCH　XCHD

★程序存储器ROM传送指令：　MOVC

★片外RAM或I/O传送指令：　　MOVX



数据传送指令操作域示意图

1.MOV指令（*只影响P标志位*）

指令格式：　　如：　MOV A,#30H

目的操作数 可以为：　A、Rn、direct、@Ri、sfr

源操作数可以为：　　Rn、direct、@Ri、#data、 sfr



```
MOV  R0 ,#30H        ;(R0) = 30H
MOV  R1 ,#31H        ;(R1) = 31H
MOV  5FH ,#0AAH      ;(5FH) = AAH
MOV  30H,#00H        ;(30H) = 00H
MOV  31H ,#01H       ;(31H) = 01H
MOV  A, R0           ;(A) = 30H
MOV  R1 , A          ;(R1) = 30H
MOV  A ,  @R0        ;(A) = 00H
MOV  30H , 31H       ;(30H) = 01H
```

```
MOV  A , 5FH                ;(A) = AAH
MOV  5FH , #5FH             ;(5FH) = 5FH
MOV  DPTR , #9000H          ; (DPH)=90H;(DPL)=00H
MOV  A , 30H                ; (A)=01H
MOV  B  ,  A                ;(B)即(F0H) = 01H
MOV  TMOD , #11H            ;(TMOD)即(89H)=11H
```

MOV指令使用注意：

（1）对SFR操作的指令中出现的常为SFR的名称，而不是它的直接地址，但其操作与直接地址相同。

（2）80H~FFH地址区间内只有21个SFR，其间许多地址无定义。对于无定义的地址不可操作，否则，欲写入存放的数据将丢失、从中读出的数据是不定数。

例 MOV A，#01H

MOV E0H ，#01H

上两条指令操作结果相同，但不是同一条指令，其目标代码不同。

对SFR中可位寻址的寄存器可进行位操作时，要求SFR的地址能被8整除。SFR中位寻址的位在指令中常以位名出现，例如ACC.0 、ACC.7…，PSW.0、 PSW.3 、P1.1…等。

（3）对于操作数中出现@Ri指令时

★ @的意义是以其内容为地址。

例 MOV 40H，#30H

MOV R0，#40H

MOV A，@R0

以R0的内容40H为地址，40H单元的内容送入A，因此执行的结果是（A） = (40H) = 30H

注意R0的内容40H不是被操作的数，而是被操作数所在地址。

★ Ri只可以是R0或R1，对R2~R7无效！

注意区分以下指令的不同操作：

MOV A，40H ；(40H) 送A

MOV A，#40H ;立即数40H送A

MOV 40H ，#40H ;立即数40H送40H单元存放

MOV 40H ，30H ;(30H)单元的内容送40H单元存放

指令中直接地址和立即数均以数据出现，以 # 区分。

传送指令运行后，被传送单元(即源数据单元)内容不变。

例 MOV A ，#01H ；

MOV R0 ，A ;运行后(R0) = (A) = 01H

MOV指令的位传送指令均与C有关

例 MOV C , bit

MOV bit , C ;bit为位地址或者位名，被传送的是该位的内容

（4）对于操作数为Rn的指令：

Rn可为R0~R7中的任何一个。片内RAM中有4个工作寄存器区，即0~3区，也就是有4组R0~R7，指令中出现的Rn是哪一区的确定于程序状态字寄存器PSW中RS1、RS0所处的组态，

PSW.4 (RS1)	PSW.3 (RS0)	
0	0	选0区
0	1	选1区
1	0	选2区
1	1	选3区

计算机上电复位时(PSW) = 00H，即默认选0区，如果程序中需要使用其它区时，则可用位操作指令改变RS1或RS0的内容，以达到切换到其它工作寄存器区的目的。

2.栈操作指令：

压入堆栈指令　PUSH

弹出堆栈指令　POP

堆栈的概念：在片内存储器内划分出一个特殊的连续区域，用以存放需保存的信息。

堆栈的建立：由堆栈指针SP的内容指出其起始位置。

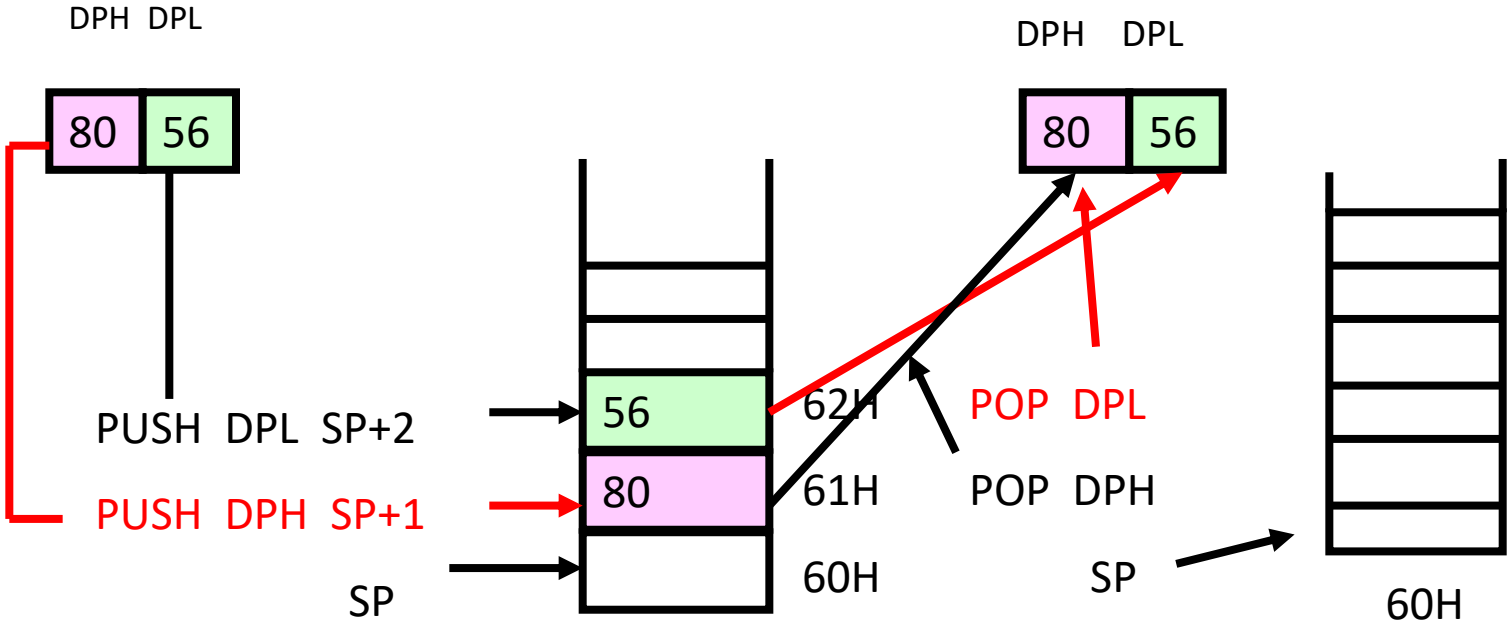
例：MOV SP，#60H

SP的内容被赋予60H，则堆栈的起始位置为60H，栈从60H开始向地址增大的方向生成。

由以下程序理解栈操作指令：

```
MOV  SP ，#60H          ;建立堆栈
MOV  DPTR ，#8056H      ;(DPH) = 80H ,(DPL) = 56H
PUSH   DPH              ;入栈存
PUSH   DPL              ;入栈存
MOV  DPTR ，#2000H      ;修改数据指针
。 。 。 。 。 。 。
POP    DPL              ;恢复DPL
POP    DPH              ;恢复DPH，(DPTR)=8056H
```

栈操作指令示意图



PUSH压入时SP先加1后压入

POP弹出时SP先弹出后减1

使用栈操作指令注意：

① PUSH、POP指令的操作遵循先进而后出的原则。

② PUSH、POP指令常成对使用，注意使用顺序。上例中若出栈顺序错误则DPTR内容不能恢复入栈前状态。

③ PUSH压入时SP先加1后压入。

④ POP弹出时SP先弹出后减1。

⑤ SP指针总是指在栈底，栈向上(向地址增大的方向)生成。还有调用子程序指令及中断系统也有栈操作，合适设置堆栈大小，防止溢出。

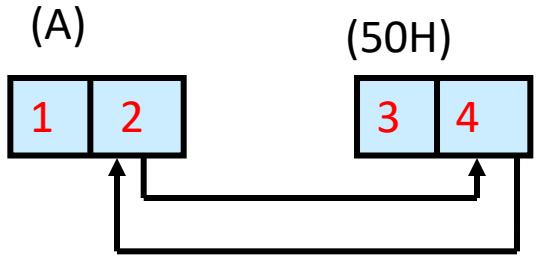
⑥ 操作数只能是 direct（包含SFR）。

3.交换指令　功能：进行数据交换。有两类：

全字节交换指令　XCH

低半字节交换指令 XCHD

例　MOV　A，#30H
　　MOV R0，#05H
　　XCH　A，R0
运行后(A)=05H；(R0)=30H



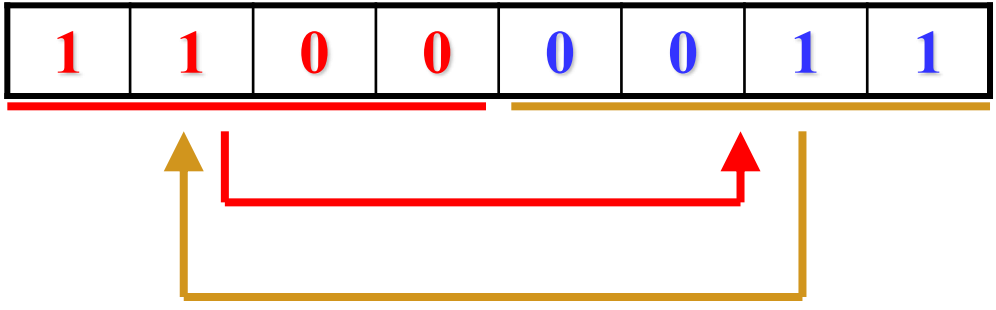
MOV A，#12H
MOV　R1，#50H
MOV 50H，#34H
　XCHD A, @R1
运行后(A)=14H　(50H)=32H

注意：该指令也称累加器A传送指令，即目的操作数只能是A，原操作数可以是：Rn、direct、@Ri、和SFR。

SWAP　高低四位互换指令

操作数只有一个，且必须为A。

已知：**A=11000011B（0C3H）**



执行完SWAP　A指令后，A 中内容为：**00111100B（3CH）**



4.程序存储器传送指令 MOVC

只有两条指令

MOVC A , @A+PC

;(A)+(PC)形成地址，再把该地址内容送A

MOVC A , @A+DPTR

;(A)+(DPTR)形成地址，再把该地址内容送A

例： 求某数 n^2

要求编一子程序，进入该子程序前 (A) = n

执行该子程序后 (A) = n^2

假定 $0 \le n \le 9$

通用的程序如下：

```
                                    ；已知（A）=n，求n²=?
SQ:     MOV   DPTR , #TABEL    ;DPTR的内容修改为平方表
        MOVC  A , @A+DPTR        的表首地址
        RET
TABEL: DB  00H，01H，04H……  ；以下略
```

注：MOVC指令目的操作数只能是A，然后从A转送别处。

5.片外数据存储器RAM传送操作　MOVX

共有4条指令：

```
MOVX   A ，@Ri          ;读指令
MOVX   A ，@DPTR        ;读指令
MOVX   @Ri ，A          ;写指令
MOVX   @DPTR ，A        ;写指令
```

@的意义仍是以其内容为地址。

注意：（1）对于以@DPTR为操作数的指令，DPTR的内容是16位地址，因此可以访问64K大小的空间（访问I/O时，也使用该指令）

（2）注意区分： MOV A，@R0； 片内RAM

MOVX A，@R0；片外RAM

两条指令源及目的相同，而操作域不同，以助记符区分。

数据传送指令综合举例

```
    ORG   8000H
MOV   SP ，#60H          ；(SP) = 60H 建立堆栈
MOV   A ，#12H           ；(A) = 12H
MOV   R0 ，#50H          ；(R0) = 50H
MOV   30H ，#03H         ；(30H) = 03H
MOV   DPTR ，#8400H      ；(DPTR) = 8400H
MOVX  @DPTR ，A          ；(8400H) = 12H
MOV   @R0 ，A            ；(50H) = 12H
XCH   A ，30H            ；(A) = 03H ，  (30H) = 12H
```

```
PUSH   DPH                ;(SP) = 61H  (61H) = 84H
PUSH   DPL                ;(SP) = 62H  (62H) = 00H
MOV    DPTR ，#TABEL      ;(DPTR) = 8600H
MOVC   A ，@A+DPTR        ;(8600H+03H)送A，(A) = 44H
POP    DPL                ;(DPL) = 00H  (SP) = 61H
POP    DPH                ;(DPH) = 84H  (SP) = 60H
MOV    30H ，A            ;(30H) = 44H
XCHD   A ，@R0            ;原(50H)=12H，（A)=42H(50H)=14H
MOVX   A ，@DPTR          ;(8400H)送A，(A) = 12H
XCH    A ，@R0            ;(A) = 14H,(50H) = 12H
       ORG    8600H
TABEL: DB     11H, 22H
       DB     33H, 44H
```

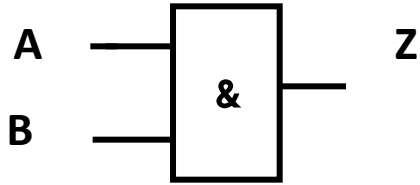
3.4 逻辑运算指令

- 3.4.1 对累加器A进行的逻辑操作
- 3.4.2 包含双操作数的逻辑运算指令
- 3.4.3 位逻辑操作指令

3.4 逻辑运算指令

关于逻辑运算法则：

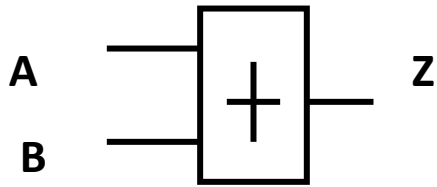
★与运算：逻辑乘　符号



记作 Z = A ×B　或　Z = A ∧B

只有输入信号均为1时输出为1。

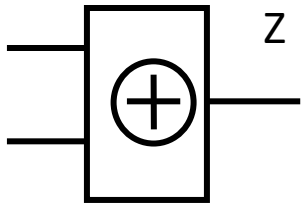
★或运算：逻辑加　符号



记作 Z = A +B　或　Z = A ∨B

输入信号中有一个1则输出为1。

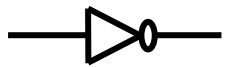
★异或运算：　　符号　A　B



记作 $Z = A \oplus B$

输入信号相同则输出为0，输入信号相异则输出为1。

★非运算：求反　　符号　$\overline{A}$



记作 $Z = \overline{A}$

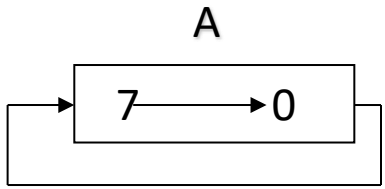
3.4.1 对累加器A进行的逻辑操作

CPL A ；A求反

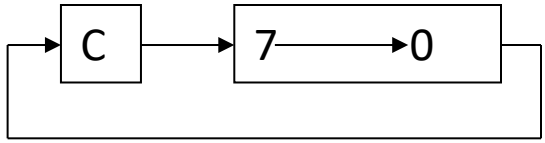
CLR A ；A清0

移位指令：

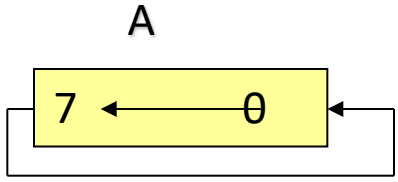
(1) RR A



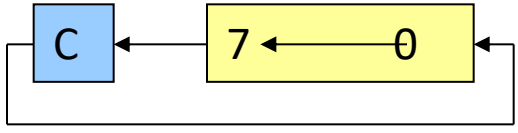
(2) RRC A



(3) RL A



(4) RLC A



移位指令助记符意义：

第一个字母R 表示循环（Recycle）移位操作。

第二个字母 表示移位方向，L 向左；R 向右。

第三个字母 表示是否通过C。为C表示连同C共9位数一起移位；无第三字母，表示与C无关。

```
例：CLR    C                ;  C = 0
    MOV    A , #0E2H        ;  1 1 1 0 0 0 1 0   E2H
    RL   A                  ;  1 1 0 0 0 1 0 1
    RLC  A                  ;  1 0 0 0 1 0 1 0   C=1
    RR   A                  ;  0 1 0 0 0 1 0 1
    RRC  A                  ;  1 0 1 0 0 0 1 0   C=1
    SWAP   A                ;  0 0 1 0 1 0 1 0
```

3.4.2 包含双操作数的逻辑运算指令

一般形式：

ANL		与
ORL	第一操作数，第二操作数	或
XRL		异或

功能： 第一操作数和第二操作数进行相应的逻辑运算，结果在第一操作数指出的地址。

Exclusive-OR: 异或

例1：MOV A，#46H

MOV 40H，#59H

ANL A，#37H

ORL 40H，A

ORL A，40H

XRL A，#55H

```
  01000110
  00110111
  ---------
  00000110   A=06H
  01011001
  ---------
  01011111   40H=5F
  00000110
  ---------
  01011111   A=5FH
  01010101
  ---------
  00010101   A=0AH
```

注：

逻辑运算结果均在第一操作数中（目的），

第二操作数内容不变。目的操作数只能是A或direct

此类逻辑运算指令用途很广，

例2： 用“与”掩码的方法获取数据中需要的部分

MOV A， #0F6H ；设A中为有符号数

① ANL A，# 0FH ；屏蔽高4位 （A）=06H

② ANL A，#7FH ；屏蔽符号位 （A）=76H

③ ANL A，#80H ；切取符号位 （A）=80H

0FH，7FH，80H，称为掩码，使用不同的掩码可屏蔽掉不同的部份，留下有用的信息。

例3： 用“或”指令,置位任何RAM单元或寄存器的某些位，方法是将需置1的位和“1”相或。

如： ORL P1 ，#00110011B；P1口的D_0，D_1，D_4，D_5位置1

例4： 用异或指令对RAM单元或寄存器某些位求反。

方法是：欲取反的位与“1”相异或

例如：XRL P1 ，#00110011B

；P1口的D_0，D_1，D_4，D_5取反。

3.4.3 位逻辑操作指令

1. ORL C , BIT ; C和某位地址的内容相或

   ANL C , BIT ; C和某位地址的内容相与

2. CLR C （BIT） ; C （BIT） 清零

   CPL C （BIT） ; C （BIT） 求反

   SETB C （BIT） ; C （BIT） 置1

位逻辑指令举例

```
MOV  24H,   #02H      ;  (24H) = 02H,(24H.1=21H位)=1
MOV  2EH,   #05H      ;  (2EH)=05H, (2EH.2=72H位)=1,
                         (2EH.0=70H位)=1
MOV  C,     21H       ;  ( C )=1
ANL  C,     72H       ;  ( C) =1
CLR  C                ;  ( C) =0
ORL  C,     24H       ;  (24H位)=0, ( C )=0
```

注：

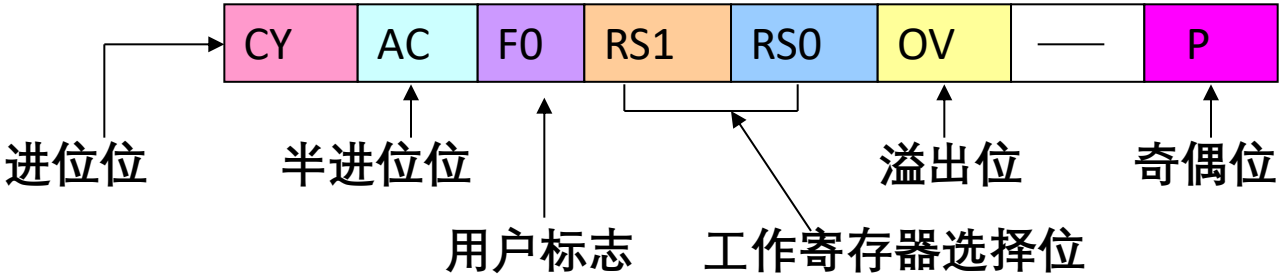
- 位逻辑指令是某一位地址或C的内容进行逻辑操作，参与运算的只是一位数。
- 片内RAM可位寻址字节地址是20H—2FH，位地址范围是00H-7FH，共128位，注意区分字节地址和位地址。
- 特殊功能寄存器SFR中可位寻址的位亦可参与位逻辑运算，这些位所在的字节地址必能被8整除，其余不可位寻址的位不可参与位逻辑运算。

3.5 算术运算指令

- 3.5.1 加法类指令
- 3.5.2 减法类指令
- 3.5.3 十进制加法调整指令
- 3.5.4 乘法指令
- 3.5.5 除法指令

3.5 算术运算指令

程序状态字寄存器PSW各位的意义



★高位进位标志位CY，简称C：

反映算术运算时，最高位D7向更高位有无进位或借位， 若有,CY = 1；若无,CY = 0 。

★半进位标志AC：

反映算术运算时，低半字节向高半字节有无进位或借位，若有AC = 1；若无AC = 0 。

```
  1 0 1 0 | 1 1 0 1
+ 1 0 1 0 | 1 0 0 0
-----------------------
1 0 1 0 1 | 0 1 0 1
```

D3位相加向D4位有进位

AC = 1　CY = 1

```
  1 0 1 0 0 | 1 0 1
- 0 0 0 1 1 | 0 0 0
-----------------------
  1 0 0 0 1 | 1 0 1
```

D3位相减向D4位有借位

AC = 1　CY = 0

★溢出标志位OV：　反映运算的结果是否发生溢出，有溢出OV = 1；无溢出OV = 0　溢出的概念：

8位有符号数表示的范围：原码：-127~+127　反码：-127~+127

补码：-128~+127　运算的结果超过此范围则发生溢出。

例　120 + 105 = 225

```
  0 1 1 1 1 0 0 0
+ 0 1 1 0 1 0 0 1
-----------------
  1 1 1 0 0 0 0 1
```

结果为负数，　显然错误！

符号位

以下将D6向D7的进位称为C6，将D7向更高位的进位称为C7。

此例中　C7=0 ,C6=1, 有溢出，OV=1

此例为数值位溢出侵犯符号位，发生错误。

溢出判断

- 带符号数的表示范围

8位二进制数，各码表示的范围如下，超出就会溢出

原码 **1111 1111B ~ 0111 1111B (-127~+127)**

FFH, FEH, …, 81H, **00H (80H)**, 01H, …7FH

-127, -126, …, -1, 0, 1, … 127

反码 **1000 0000B ~ 0111 1111B (-127~+127)**

80H, 81H, …, FEH, **00H (FFH)**, 01H, …7FH

-127, -126, …, -1, 0, 1, … 127

补码 **1000 0000B ~ 0111 1111B (-128~+127)**

80H, 81H, …, FFH, **00H**, 01H, …, 7EH, 7FH

-128, -127, …, -1, 0, 1, …, 126, 127

又例 -3 + （-2） = -5

	-3	-2
原码	1 0 0 0 0 0 1 1	1 0 0 0 0 0 1 0
反码	1 1 1 1 1 1 0 0	1 1 1 1 1 1 0 1
补码	1 1 1 1 1 1 0 1	1 1 1 1 1 1 1 0

-3 + （-2）

1 1 1 1 1 1 0 1

+ 1 1 1 1 1 1 1 0

1 1 1 1 1 1 0 1 1　　-5的补码

此例中　C7=1 ， C6=1 ,OV=0, 无溢出，结果正确。

总结溢出规律：

C7 = 0 C6 = 0 数值位、符号位均无溢出，OV = 0

C7 = 0 C6 = 1 数值位溢出侵犯符号位， OV = 1

C7 = 1 C6 = 0 符号位有溢出， OV = 1

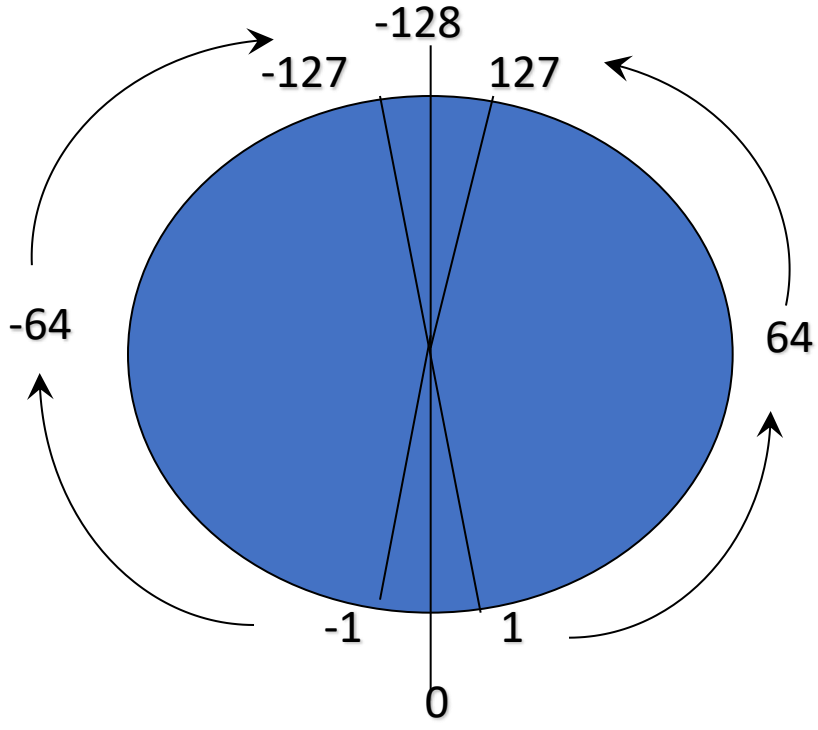
C7 = 1 C6 = 1 数值位符号位均有溢出但结果正确，OV = 0

C7和C6相同OV为0，相异OV为1，符合异或运算法则：

$$OV = C7 \oplus C6$$

注意：溢出OV与进位CY的区别。

CY主要用于无符号数的运算，OV主要用于有符号数的运算。


-128
-127
127
-64
64
-1
1
0

★奇偶标志位P：

反映累加器A的内容中1的个数为偶数或奇数，

偶数个1 P = 0 ；奇数个1 P = 1

★工作寄存器区选择位RS1、RS0

改变RS1、RS0的组态，可切换Rn的不同工作区。前述。

★用户标志F0：可视实际需要规定其意义。

3.5.1 加法类指令：

（1）ADD A，第二操作数；（A）+（第二操作数）送A

（2）ADDC A，第二操作数；（A）+(第二操作数)+(C)送A

该指令影响PSW中CY， AC， OV， P标志位。

例：MOV A，#05H　　0 0 0 0 0 1 0 1　C=0 AC=1 C6=0

ADD A，#3BH　　+ 0 0 1 1 1 0 1 1　OV=0 P=1

0 1 0 0 0 0 0 0

ADD A，#0F5H　+ 1 1 1 1 0 1 0 1　C=1 AC=0 C6=1

1　0 0 1 1 0 1 0 1　OV=0 P=0

ADDC A，#70H　　+ 0 1 1 1 0 0 0 0

+ 1 0 1 0 0 1 0 1　C=0 AC=0 C6=1

1 0 1 0 0 1 1 0　OV=1 P=0

(3) 自增1指令 INC 第一操作数；

指定单元内容加1，结果在原单元。

例： INC A ； （A）+1 送 A

INC R_0 ； （R_0）+1 送 R_0

注：该指令不影响PSW中标志位，例:

INC A ；分别运行这两条指令后，A的内容都加1

ADD A，#01H ；但运行后PSW的内容不同。

3.5.2 减法类指令

（1）SUBB A，第二操作数；带借位减法指令

（A）-（第二操作数）-（C）送 A

该指令影响PSW中CY， AC， OV， P标志位。

例： SETB C

MOV A，#73H

SUBB A，#86H ； (A) = ECH CY=1 AC=1

1 01111 0011 C6=0 OV=1 P=1

- 10000110

11101101 再减C得： 11101100

注：

减法指令没有不带借位的指令，注意使用该指令前C的内容，若不需减C，则用CLR C 指令清C后再作SUBB 。

ADD，ADDC，SUBB 指令中，一个加数或被减数一定先存入A，且结果也在A里。

（2）自减1指令 DEC 第一操作数；指定单元内容减1，结果在原单元。

例：DEC A ；（A）—1送 A

注：该指令不影响PSW中标志位。

加减法指令举例：

例　MOV R0，#50H；　　5 5

MOV A，#55H；　+　7 F

ADD A，#7FH；　　D 4　　C=0

MOV @R0, A;　　(50H)=D4H

INC R0;　　D 4　　(R0) = 51H

ADD A,#70H;　　+ 7 0　　C = 1

MOV @R0, A;　　1 4 4　(51H) = 44H

INC R0;　　4 4　　(R0) = 52H

ADDC A,#22H;　　+ 2 2

MOV　@R0 ,A;　　6 6　再加C得　（A）=67H

(52H)=67H　　C = 0

```
INC  R0                    (R0) = 53H
SUBB  A,#89H;              6 7
MOV  @R0 ,A;             - 8 9
INC  R0;                   D E    C = 1  (53H)=DEH  (R0)=54H
SUBB  A,#0AFH;             D E
MOV  @R0, A;             - A F      C=0
                           2 F  再减C得2EH      (54H)=2EH
```

3.5.3 十进制加法调整指令 DA A

用于BCD码相加后结果的十进制调整。

为什么要调整？ 例 96 + 89=185

```
  1 0 0 1 0 1 1 0
+ 1 0 0 0 1 0 0 1
-----------------
1 0 0 0 1 1 1 1 1
+ 0 1 1 0 0 1 1 0
-----------------
  1 0 0 0 0 1 0 1
```

调整的原则：

1. 以半字节（4位）为单位，过9加6或60
2. AC=1时低半字节加6
3. CY=1时高半字节加60
4. AC=1 CY=1时，加66

例 19 + 58 = 77

```
MOV  A , #19H      0 0 0 1 1 0 0 1
ADD  A , #58H    + 0 1 0 1 1 0 0 0
DA   A             0 1 1 1 0 0 0 1
MOV  50H , A     + 0 0 0 0 0 1 1 0
                   0 1 1 1 0 1 1 1
```

注:

①DA A 指令只能用于加法指令之后，对减法指令后的结果不能调整。

② 本指令是根据A的原始值和PSW的状态，对A进行加06H，60H 或66H的操作。

③ 参与相加的二数均为组合BCD码（压缩BCD码），调整后得到正确的BCD码结果，不是将二或十六进制数的加法运算调整成十进制。

④本指令对A的BCD码加法结果进行调整，两个压缩BCD码按二进制数相加之后，必须经本指令调整才能得到压缩型BCD码的和数。

3.5.4 乘法指令 MUL AB（51系列只有无符号乘除法）

将A，B两寄存器中的两个无符号8位二进制数相乘，积为16位，其低8位存放于A，高8位存放于B。

若积不大于8位（即B的内容为0），则OV=0，否则OV=1

乘法指令后，CY=0，AC不变。

例　16×9 = 144

```
MOV   A , #10H
MOV   B , #09H
MUL   AB   ; (A) = 90H (144D), (B) = 00H
             (OV) = 0 ,(CY) = 0,  AC不变
```

又例　80×160 = 12800

```
MOV   A ，#50H
MOV   B ，#0A0H
MUL   AB     ;(A) = 00H ，(B) = 32H
        ∵B≠0  (OV) = 1,(CY) = 0,AC不变
```

3.5.5 除法指令　DIV AB

操作：

A B中均为无符号整数，A除以B，商的整数部分存于A，商的余数部分存于B。

运行后，CY=0 ；若除数B为0(非法），则商为不定值，且OV=1；若B不为0，则OV=0

例　251÷18 = 13 余17

```
MOV   A , #251   ;#FB
MOV   B , #18    ;#12H
DIV   AB         ;(A) = 0DH,(B) = 11H,
                  (OV) = 0,(CY) = 0
```

3.6 控制转移指令

- 3.6.1 无条件转移指令
- 3.6.2 条件转移指令
- 3.6.3 其它指令
- 3.6.4 控制转移指令综合举例

3.6 控制转移指令

可使程序转向非顺序单元执行，使程序发生分支或循环。

3.6.1 无条件转移指令

1.调用与返回指令

(1)绝对调用指令 ACALL addr11

调用子程序指令。该指令可改变程序计数器PC的低11位地址，所调子程序首地址必须在2K页内。

(2)长调用指令 LCALL addr16

调用子程序指令。所调子程序首地址可在64K地址的任何位置。

(3)从子程序返回指令 RET

2.无条件跳转指令

(1)绝对跳转指令 AJMP addr11

可使程序在2k页内跳转，即可改变低11位地址。

(2)长跳转指令 LJMP addr16

可使程序跳转到64K程序存储器的任何位置。

(3)相对短跳转指令 SJMP rel

其中rel为偏移量，rel的数字指出PC跳向的地址，可以向上或向下跳转，rel和当前PC之差的范围是-128 ~ +127，差为负值时表示从大地址向小地址跳转，为正值时表示从小地址向大地址跳转。

直接给出地址情况较少，通常使用标号地址。如下：

例：

```
        8000H   MAIN:   MOV   A , #01H
        8002H           ADD   A , #02H
                        …
        8006H           SJMP  MAIN  (DIR)
PC ——→  8008 H          MOV   A , @R0
                …
        800FH   DIR:    MOV   A , R2
```

偏移量rel的计算公式为(e) = 目的地址 – PC当前值，注意用补码表示。PC当前值为跳转指令下面一条指令的地址，即8008H。

跳转到MAIN时：(e) = 8000H – 8008H = F8H (-8)的补码

跳转到DIR时：(e) = 800FH – 8008H = 07H

- **（4）间接转移指令**

  JMP @A+DPTR

  （PC）←（A）+（DPTR）

这是一条极其有用的**多分支选择转移指令**，由**DPTR决定多分支转移程序的首地址**，由**A的内容**来动态**选择**其中的**某一分支转移指令**。从而一条间接转移指令就代替了众多的转移指令。

- 设A中内容为0，2，4，6中的一个，下面的指令序列将转移到以JMP-TBL标号为起始地址的转移表中的4个AJMP指令之一处。

```
        MOV  DPTR，#JMP-TBL；起始地址送DPTR
        JMP @A+ DPTR; 程序转向（A）+（DPTR）为地址的单元
        。 。 。 。 。 。
JMP-TBL：  AJMP  LABEL0；
           AJMP  LABEL1；
           AJMP LABEL2；
           AJMP LABEL3；
```

转移类指令注意事项：

★ 调用与返回指令常成对使用，返回指令RET应出现在每一个子程序的末尾。

★ 调用与返回指令都要有栈操作，使用该类指令前要建立堆栈，以便保护断点。

★ 转移指令与调用子程序指令的异同：

相同：都改变PC值，使程序转入非顺序单元执行。

不同：调用指令在子程序执行结束后一定返回原断点地址，因此一定有栈操作。

转移指令跳转后不一定回到原断点，因此无需堆栈保存断点地址。

3.6.2 条件转移指令

该类指令是当符合某种条件时，程序转移到另一地址执行，不符合这种条件时，程序按原顺序继续执行。

1.判零相对转移指令 累加器A的内容是否为0作为转移条件

JZ rel ;A的内容为0则转移，否则顺序执行。

JNZ rel ;A的内容非0则转移，否则顺序执行。

rel偏移量和前面类似(8位,可以是直接地址也可以是标号地址)

例
```
MOV  A , #56H
SUBB A , R0
JZ   next    ;当减法结果为0时跳到next地址执行
...          否则继续往下执行。
```

2.比较相对转移指令 (三字节指令)

CJNE （目的数），(源数)，rel

操作：将目的操作数内容与源操作数内容相比较，

若不等则相对转移，

若相等则顺序执行。

若(目的字节)＜(源字节)则C = 1，否则C = 0;同时，该指令执行后两操作数的内容都不改变。

例: CJNE A，#04H，MAIN ;(A) ≠04H则跳转到MAIN标号

(A) = 04H顺序执行

如果（A)＜04H则C= 1

CJNE @R0，#05H，DIR ;((R0))≠05H则跳DIR标号

CJNE A，30H，STOP ;(A)≠(30H)则跳STOP标号

CJNE指令具有四种格式:

CJNE A , #data , rel　　**CJNE A, direct , rel**

CJNE Rn , #data , rel　　**CJNE @Ri , #data , rel**

3.循环转移指令　DJNZ　操作数，rel

操作：操作数内容减1，结果非0则转移，结果为0则顺序执行。

第一操作数可以是Direct或Rn；该指令不影响任何标志位。

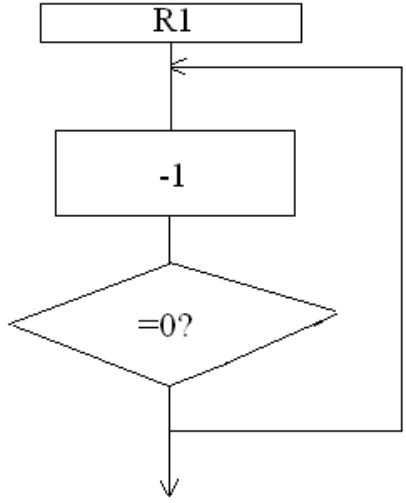
例　DJNZ　R0 ，LOOP　;(R0) – 1结果非0转LOOP标号

　　DJNZ　30H ，SQ　　;(30H) – 1结果非0转SQ标号

例1：　　指令周期见附录

```
        MOV R1，＃40
NEXT：  DJNZ R1，NEXT
        RET
```

t=[1+2*40+2]*T机器周期时间




```
例2：    MOV R1，＃50        ；1
NEXT1：  MOV R2，＃40        ；1
NEXT2：  DJNZ R2，NEXT2      ；2
         DJNZ R1，NEXT1      ；2
         RET                 ；2
```

t= [1+(1+(2*40)+2)*50+2]*T机器周期时间

4.以位地址的内容为判断条件的转移指令

① JC rel ;(C) = 1跳转;(C) = 0顺序向下执行

② JNC rel ;(C) = 0跳转;(C) = 1顺序向下执行

③ JB bit , rel ;(bit) = 1跳转;(bit) = 0顺序问下执行

④ JNB bit , rel ;(bit) = 0跳转;(bit) = 1顺序问下执行

⑤ JBC bit , rel ; (bit) = 1跳转，且将该位内容清0;
(bit) = 0顺序向下执行

3.6.3 其它指令

1.RETI 从中断返回指令，中断系统中详细介绍。

2.NOP 空操作指令。执行该指令CPU不作任何操作，只是占用一个机器周期的时间。

3.6.4 控制转移指令综合举例

例1　判断P1口输入的数据，若为负则转FS，否则转ZS。

```
8000H  E5  90      ZS：MOV  A，P1
8002H  33                RLC  A
8003H  40  e1            JC  FS
8005H  80  e2            SJMP  ZS
8007H                    。 。 。 。 。 。
800AH              FS: ……
```

求偏移量：e =〔目的地址－下条指令地址〕补码

e1 =〔800A H - 8005H〕补码=05H

e2 =〔8000H－8007H〕补码=〔-7〕补码=F9H

例2： 判断20H单元内容中1的个数。

```
        MOV  A，20H
        MOV  R1，  #08H      ；计数判断8次
        MOV  R0 ，#00H       ；1的个数计数单元清0
LOOP：RLC   A
        JNC   ZERO;  ( C )=0,跳；=1，顺序执行
        INC   R0
ZERO：DJNZ   R1 ，LOOP
STOP：SJMP    STOP；暂停
```

3.7 汇编语言、伪指令

3.7.1 汇编语言的结构

LOOP：ADD　A，　30H　；　（A）+（30H）送 A

标号　　操作码　操作数　　；　　注释

3.7.2　伪指令

指令能够汇编成目标代码，能够被计算机识别和执行。

伪指令是用来控制汇编的，不产生目标代码，也不能被计算机执行。

（1）ORG xxxxH　　起点指令（常为双字节数据）

规定该指令后的程序或数据块的起始地址，例 ORG 8100H

（2） DB 定义字节

例： ORG 2100H

FIRST：DB 01，32，83（可以是二进制、十进制、十六进制）还可以如下定义：

FITST: DB 01

DB 32

DB 83

其中ORG指定以下数据块从2100H开始存放（FIRST），DB指定2100H开始的单元存放01，32，83

要取定义好的数据的话，就需要前面介绍的**MOVC**指令，具体方法是：

```
MOV DPTR， # FIRST1
MOV A， # NN ；取 数01 NN＝0；取数32 NN＝1；83 NN＝2
MOVC A,@A+DPTR
```

（3） DW 定义十六位数据

例　　ORG 8000H

　　DW 7603H，7589H

则：　（8000H）=76H　　（8001H）=03H

　　（8002H）=75H　　（8003H）=89H

（4） DS　定义若干空单元

例：　　ORG 5000H

　　DS 07H

　　MOV A，#7AH

　　ADD A，#22H

从5000H开使预留出七个空单元以备后用，这样下面的指令MOV A，#7AH 则从5007H开始存放。

（5） EQU　赋值指令（用于变量赋值）

例： MAA EQU 30H ； 则MAA与30H等价。

如：MOV MAA, # 44H

MOV 30H， # 44H ;功能相同

再如: 已知 (30H)= 50H

MOV A, #MAA ;A=30H

MOV A,MAA ;A=50H

LJMP MAA ; PC=50H

主要功能在于给一个表达式或者字符串命名,相当于变量赋值.

EQU只能给标号一次赋值，整个程序中再不能更改。

如: LOOP EQU 8300H ； LOOP的地址为8300H

往8300H单元写入数据 50H，指令如下：

MOV A， # 50H

MOV DPTR， # LOOP

MOVX @DPTR，A

注意8位和16位地址的区别

16位地址相当于一个标号。

EQU也可以用“ = ”来替代，功能相同。

（6）DATA 和XDATA

两指令主要功能是为RAM单元命名，DATA是针对内部RAM，而XDATA是针对外部RAM。和EQU功能类似，了解即可。

如： JIEGUO DATA 50H

KAI XDATA 2040H

（7）Bit 功能是为位地址单元起名，类似于EQU，不同的是一个二进制位。

如： XIAO BIT 20H
CG BIT PSW.5

（8） END 汇编语言程序结束标志

汇编时，遇到该指令后面的程序和数据将不再生成目标代码。

（9）DL 定义标号值

COUNT1 DL 2300H； 定义COUNT1 =2300H

。。。。

COUNT2 DL COUNT1 +1；定义COUNT2=2300H+1

DL可在同一源程序中，对同一标号多次赋值。

复习要点

- 寻址方式
- 111条指令
- 堆栈
- A
- 访问 存储器 指令（片内RAM，片外RAM，ROM）
- 指令写法是否正确?
- 常用伪指令